Zephyr Project

IT-Schulungen.com
Technology & Transformation

Konfiguration mit Kconfig

Erklärung, Demo und Recherche

1.11

Vom gewünschten zum wirksamen Wert

Fünf Schritte zur überprüfbaren Konfiguration

01

Verstehen

Softwarefunktionen und CONFIG_-Symbole einordnen

02

Anfordern

Werte über prj.conf und weitere Fragmente setzen

03

Auflösen

Defaults und Abhängigkeiten auswerten lassen

04

Prüfen

Wirksame Werte in build/zephyr/.config finden

05

Nachweisen

Neu bauen und Laufzeitverhalten beobachten

Ergebnis: Eine Option wird aktiviert, in .config gefunden und über ihre Abhängigkeiten erklärt.

Was Kconfig entscheidet – und was nicht

SOFTWARE

Kconfig

Kernel- und Subsystemfunktionen

Treiberoptionen

Logging und Shell

Speicher- und Threadparameter

KONFIGURATION

Symbole und Regeln

CONFIG_-Werte

Typen und Defaults

Abhängigkeiten

sichtbare Menüs

HARDWARE

Devicetree

Geräte und Instanzen

Adressen und Interrupts

Pins und Eigenschaften

Hardwaretopologie

Merksatz: Kconfig aktiviert Softwarefunktionen; Devicetree beschreibt Geräte und Eigenschaften.

Symbole besitzen Typ und Regeln

Kconfig

```
config APP_PERIOD_MS
  int "Ausgabeintervall in ms"
  range 100 10000
  default 1000
  depends on PRINTK
```

01 Symboltypen

bool, tristate, int, hex und string bestimmen zulässige Werte.

02 CONFIG_-Präfix

In .conf und .config erscheint das Symbol als CONFIG_APP_PERIOD_MS.

Ein Symbol besteht nicht nur aus einem Wert: Typ, Prompt, Default und Abhängigkeiten wirken zusammen.

Viele Quellen ergeben eine .config

KONFIGURATIONSFLUSS

```
Board- und SoC-Defaults
        +
Subsystem- und Modulregeln
        +
prj.conf und weitere Fragmente
        ↓
build/zephyr/.config
```

01 Eingaben

prj.conf formuliert die Wünsche der Anwendung.

02 Auflösung

Kconfig prüft Typen, Defaults und Abhängigkeiten.

03 Ergebnis

.config enthält die tatsächlich wirksamen Werte.

Gewünschter Wert und wirksamer Wert können sich unterscheiden – .config ist der Nachweis.

Regeln mit unterschiedlicher Wirkung

`default`	liefert einen Wert, solange keine stärkere Zuweisung greift
`depends on`	macht ein Symbol nur wirksam oder sichtbar, wenn Bedingungen erfüllt sind
`select`	erzwingt ein anderes Symbol und kann dessen eigene Abhängigkeiten umgehen
`imply`	schlägt ein anderes Symbol schwächer vor und lässt Überschreibungen zu
`Prompt und Menü`	bestimmen Sichtbarkeit und Bedienbarkeit in menuconfig

Vorsicht mit select: Es ist kein Ersatz für korrekt modellierte Abhängigkeiten.

menuconfig gezielt nutzen

SUCHEN · VERSTEHEN · ÄNDERN

menuconfig ist ein Diagnosewerkzeug – nicht nur eine komfortable Checkbox-Oberfläche.

Öffnen	west build -t menuconfig verwendet den aktuellen Build-Zustand
Suchen	Mit / nach Symbolnamen, Prompt oder Teilbegriff suchen
Abhängigkeiten	Anzeige erklärt, warum ein Symbol unsichtbar oder nicht wählbar ist
Ändern	Wert testweise setzen und die resultierende Konfiguration prüfen
Dauerhaft machen	Beabsichtigte Einstellung in prj.conf oder ein Fragment übernehmen

Nicht nur klicken: Symbolname, direkte Abhängigkeiten und wirksamen Wert dokumentieren.

Änderung prüfen und nachweisen

TERMINAL

```
$ west build -t menuconfig
$ grep '^CONFIG_LOG=' build/zephyr/.config
$ west build
$ west build -t run


# Warnungen im Build-Log zuerst lesen
```

01 Warnung

Symbol, Wunschwert und Bedingung zuordnen.

02 Laufzeitnachweis

Die aktivierte Funktion muss sich im Verhalten oder in der Ausgabe zeigen.

Nachweis = prj.conf-Eingabe, wirksamer .config-Wert und beobachtbares Laufzeitergebnis.

Kconfig verstanden

ERFOLG

Option nachgewiesen

Symbol gefunden
Typ erkannt
Abhängigkeiten erklärt
.config geprüft

DIAGNOSE

Wunsch ≠ Wirkung

Warnung zuerst lesen
Abhängigkeit suchen
wirksamen Wert prüfen
Laufzeit vergleichen

NÄCHSTER SCHRITT

Devicetree 1.12

Geräte beschreiben
Eigenschaften lesen
native_sim-Modell prüfen
Kconfig klar abgrenzen

Lernziel: Aktiviert eine Option, findet sie in .config und erklärt ihre Abhängigkeiten.